

Projeto: Pulse Care

Grupo 48:

Nome: Luís Felipe Garcia Menezes - **RM:** 371055

1. Introdução

Descrição do Problema

Uma rede de clínicas e consultórios médicos identificou que o processo de agendamento de consultas, até então feito de forma manual e fragmentada entre recepção, enfermagem e corpo clínico, vinha gerando conflitos de horário, falta de lembretes aos pacientes e ausência de um histórico centralizado das consultas realizadas. Diante disso, a rede decidiu investir em uma plataforma digital única, capaz de centralizar o cadastro de profissionais e pacientes, o agendamento de consultas e a comunicação de lembretes, ao mesmo tempo em que mantém um histórico consultável de todo o atendimento prestado. Devido à necessidade de escalar cada parte do sistema de forma independente, agendamento, notificação e histórico possuem cargas e frequências de acesso muito diferentes entre si, optou-se por uma arquitetura de microsserviços desde a concepção do projeto.

Objetivo do Projeto

Desenvolver uma solução backend baseada em **microsserviços** utilizando Spring Boot, responsável pelo cadastro de usuários (médicos, enfermeiros e pacientes) e pelo agendamento de consultas médicas, com foco em desacoplamento entre os serviços, comunicação assíncrona via mensageria, autenticação centralizada e segurança dos dados. A solução deve atender aos requisitos funcionais do sistema cadastro, agendamento, notificação e histórico, e demonstrar a aplicação de boas práticas de arquitetura distribuída, como database-per-service, comunicação orientada a eventos e autenticação compartilhada entre serviços independentes.

2. Arquitetura do Sistema

Diferente das fases anteriores do desafio, em que a aplicação era construída como um único serviço (em arquitetura em camadas ou Clean Architecture), o PulseCare foi projetado como um conjunto de **três microsserviços independentes**, cada um com seu próprio banco de dados, mais um módulo compartilhado (pulse-care-core) com as peças comuns entre eles (segurança JWT, exceções de domínio e contrato de mensageria). Essa decomposição permite que cada serviço evolua, escale e seja implantado de forma independente.

Para simplificação foi utilizado um mono-repositorio com módulos para cada microsserviços.

Os serviços se comunicam de duas formas:

- **Síncrona (HTTP/REST)**: apenas o agendamento-service expõe endpoints REST consumidos diretamente pelo cliente e o historico-service expõe uma API GraphQL.
- **Assíncrona (RabbitMQ)**: o agendamento-service publica eventos de consulta criada/editada, consumidos de forma independente pelo notificacao-service e pelo historico-service, sem que o publicador conheça seus consumidores.

2.1 Estrutura da Aplicação

Os serviços com API REST (agendamento-service) seguem uma arquitetura em camadas:

- **Controller** - recebe as requisições HTTP, valida a entrada com Jakarta Validation (@Valid) e delega para a camada de serviço.
- **Service** - contém as regras de negócio, orquestra chamadas a repositórios e publica eventos de domínio quando aplicável. Usa @Transactional para garantir consistência.
- **Validation** - validações de negócio que não cabem em anotações de Bean Validation (ex.: e-mail duplicado, senha atual inválida).
- **Repository** - interfaces que estendem JpaRepository, com consultas customizadas via JPQL.
- **Mapper** - utiliza MapStruct para conversão entre Entity, DTO (saída) e VO

(entrada), evitando expor entidades JPA diretamente na API.

- **Exception** — tratamento centralizado de erros com `@RestControllerAdvice`, seguindo o padrão **Problem Details (RFC 7807)**.

O `historico-service` segue o padrão **CQRS** (Command Query Responsibility Segregation): ele não recebe comandos de escrita via REST, sua única fonte de dados é o evento assíncrono recebido do `agendamento-service`, e expõe apenas consultas de leitura, via GraphQL.

2.2 Módulo Compartilhado - pulse-care-core

Biblioteca compartilhada entre os três serviços, sem uma aplicação Spring Boot própria. Concentra:

- **Segurança JWT** (`JwtUtil`, `JwtAuthFilter`) - geração e validação do token, reaproveitado por todos os serviços que expõem endpoints protegidos.
- **Exceções de domínio** (`BusinessException`, `NotFoundException`) - mapeadas para respostas HTTP padronizadas em cada serviço.
- **Contrato de mensageria** (`RabbitConstants`, `ConsultaEventoDTO`) - nomes de exchange/filas e o formato do evento trocado entre os serviços, garantindo que publicador e consumidores concordem sobre o payload sem acoplamento direto de código.

2.3 Comunicação Assíncrona entre Microserviços (RabbitMQ)

O `agendamento-service` publica um evento sempre que uma consulta é criada ou atualizada, em um **exchange do tipo topic**. Os demais serviços declaram suas próprias filas e se inscrevem nesse exchange, sem qualquer acoplamento com o publicador, o `notificacao-service` e o `historico-service` podem ser adicionados, removidos ou reiniciados sem que o `agendamento-service` precise saber de sua existência. Detalhes da topologia estão na seção 6.

2.4 Containerização

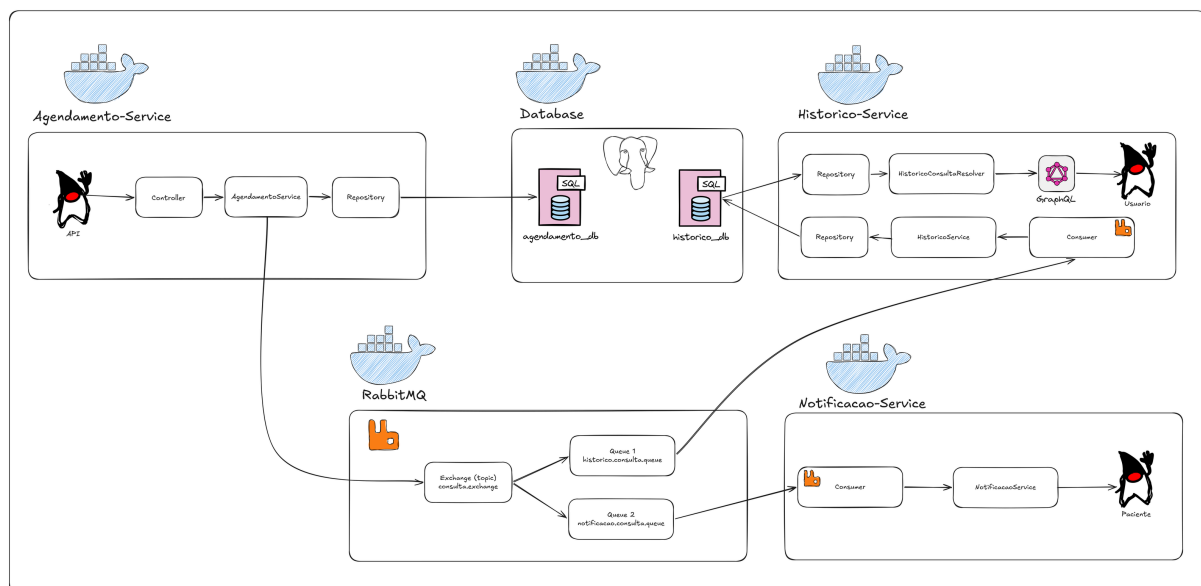
Toda a stack é executada via **Docker Compose**, com cinco containers:

- `postgres` (um único container com dois databases)
- `rabbitmq`
- `agendamento-service`

- notificacao-service
- historico-service

Todos os serviços Spring Boot são construídos a partir de um único Dockerfile multi-stage, parametrizado pelo build-arg SERVICE_MODULE, o que evita duplicar a definição de build para cada módulo. A comunicação entre os containers ocorre via rede interna do Docker, usando o nome do serviço como hostname (ex.: postgres, rabbitmq); cada serviço só inicia depois que suas dependências estão saudáveis (healthcheck).

2.5 Diagrama da Arquitetura



3. Modelagem do Domínio e Relacionamentos

Seguindo o princípio de **database-per-service**, os dados estão distribuídos em dois bancos PostgreSQL independentes: `agendamento_db` (fonte da verdade, alimentada via REST) e `historico_db` (projeção de leitura, alimentada via evento assíncrono).

3.1 Entidades Principais

`agendamento_db`

- **Tipo Usuário** - representa o papel do usuário na plataforma (MEDICO, ENFERMEIRO, PACIENTE). Tabela de domínio fixo, semeada via migration.
- **Usuario** - representa médicos, enfermeiros e pacientes cadastrados no sistema. Contém nome, e-mail (único), senha (armazenada com hash BCrypt), tipo de usuário e data de criação.
- **Consulta** - representa uma consulta médica agendada entre um paciente e um médico, com data/hora, status (ex.: AGENDADA, CONFIRMADA), data de criação e de última atualização.

`historico_db`

- **HistoricoConsulta** - projeção de leitura desnormalizada de uma consulta (inclui nome do paciente e do médico já resolvidos, sem necessidade de *join*), mantida em sincronia com o `agendamento_db` por meio dos eventos publicados no RabbitMQ.

3.2 Relacionamentos

- Um **Tipo de Usuário** pode estar associado a vários **Usuários**.
- Uma **Consulta** referencia exatamente um **Usuário** como paciente e um **Usuário** como médico.
- Um **Histórico de Consulta** corresponde a exatamente uma **Consulta** do `agendamento_db` (vínculo lógico via `consultaid`, não por chave estrangeira, os bancos são independentes).

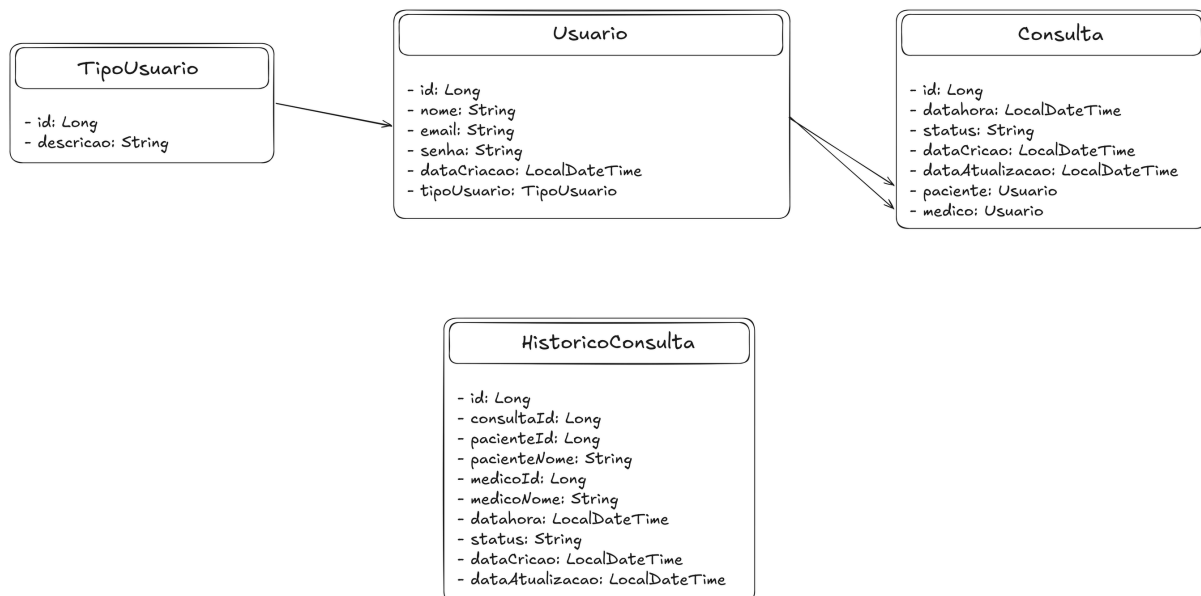
3.3 Regras de negócio refletidas

Entre as principais regras implementadas no domínio estão:

- Não é permitido cadastrar usuários com e-mail duplicado.
- A senha deve atender a critérios mínimos de segurança (mínimo de 8 caracteres, com maiúscula, minúscula, número e caractere especial).
- Uma consulta só pode ser agendada ou atualizada para uma data/hora no futuro.
- Apenas usuários do papel MEDICO podem cadastrar novas consultas, MEDICO e ENFERMEIRO podem editá-las, os três papéis podem consultá-las.
- A listagem de consultas exige informar exatamente um filtro (pacienteId **ou** medicId).
- O histórico de consultas é somente leitura, nunca é alterado diretamente, apenas reflete os eventos recebidos do agendamento-service.

3.4 Diagramas

Abaixo está o diagrama de entidades e relacionamentos do sistema:



4. Autenticação e Autorização (JWT)

Diferente das fases anteriores (monolíticas, com um único ponto de autenticação natural), a arquitetura distribuída do PulseCare exigiu uma estratégia explícita de autenticação compartilhada entre serviços independentes.

Existe **um único endpoint de login**, exposto pelo agendamento-service:

POST /auth/login

O token JWT gerado é assinado com um segredo compartilhado (via pulse-care-core) e é válido para autenticar requisições em **todos os módulos**, inclusive no historico-service, que não possui endpoint de login próprio. Isso evita duplicar lógica de autenticação em cada serviço, mantendo apenas a validação do token (via JwtAuthFilter) distribuída entre eles.

O papel do usuário é embutido no token (*claim* role) e controla o acesso ao recurso de consultas:

Papel	Permissão sobre /consultas
MEDICO	Cria e edita
ENFERMEIRO	Edita
PACIENTE	Lista/consulta

Os demais recursos (/usuarios, /tipos-usuario) exigem apenas um token válido, sem restrição adicional de papel.

5. Descrição dos Endpoints

5.1 Tabela de Endpoints REST

Endpoint	Método	Descrição
/auth/login	POST	Realiza autenticação e retorna o token JWT
/usuarios	GET	Lista usuários com paginação
/usuarios/{id}	GET	Busca usuário por ID
/usuarios	POST	Cadastra um novo usuário
/usuarios/{id}	PUT	Atualiza dados do usuário
/usuarios/{id}/senha	PUT	Altera a senha do usuário
/usuarios/{id}	DELETE	Remove usuário
/tipos-usuario	GET	Lista os tipos de usuário disponíveis
/tipos-usuario/{id}	GET	Busca tipo de usuário por ID
/consultas	POST	Agenda uma nova consulta (restrito a MEDICO)
/consultas/{id}	PUT	Atualiza uma consulta (restrito a MEDICO/ENFERMEIRO)
/consultas/{id}	GET	Busca consulta por ID
/consultas?pacienteld= ou medicold=	GET	Lista consultas de um paciente ou médico

5.2 API GraphQL - historico-service

Endpoint único: POST /graphql - console interativo em /graphql (também protegido por JWT).

```
type HistoricoConsulta {  
  id: ID!  
  consultaId: ID!  
  pacienteId: ID!  
  pacienteNome: String!  
  medicoId: ID!  
  medicoNome: String!  
  dataHora: String!  
  status: String!  
}  
  
type Query {  
  historicoPorPaciente(pacienteId: ID!): [HistoricoConsulta]  
  consultasFuturas(pacienteId: ID!): [HistoricoConsulta]  
}
```

5.3 Exemplos de Requisição e Resposta

Login

POST /auth/login

Request:

```
{  
  "email": "medico.seed@pulsecare.com",  
  "senha": "Senha123!"  
}
```

Response (200):

```
{  
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQ..."  
}
```

Cadastrar Usuário

POST /usuarios *(requer token válido)*

Request:

```
{
  "nome": "João da Silva",
  "email": "joao.silva@pulsecare.com",
  "senha": "SenhaForte123!",
  "tipoUsuariold": 3
}
```

Response (201):

```
{
  "id": 42,
  "nome": "João da Silva",
  "email": "joao.silva@pulsecare.com",
  "tipoUsuario": {
    "id": 3,
    "descricao": "PACIENTE"
  },
  "dataCriacao": "2026-09-14T10:30:00"
}
```

Cadastrar Consulta

POST /consultas (*requer papel MEDICO*)

Request:

```
{
  "pacienteld": 9003,
  "medicold": 9001,
  "dataHora": "2027-06-15T10:00:00",
  "status": "AGENDADA"
}
```

Response (201):

```
{
  "id": 9001,
  "pacienteld": 9003,
  "pacienteNome": "Paciente Teste",
  "medicold": 9001,
  "medicoNome": "Medico Teste",
  "dataHora": "2027-06-15T10:00:00",
}
```

```
"status": "AGENDADA",  
"dataCriacao": "2026-09-14T10:30:00",  
"dataAtualizacao": null  
}
```

Exemplo de erro (RFC 7807 / Problem Details)

Response (400):

```
{  
  "type": "about:blank",  
  "title": "Bad Request",  
  "status": 400,  
  "detail": "Email já cadastrado: joao.silva@pulsecare.com",  
  "instance": "/usuarios",  
  "timestamp": "2026-09-14T10:31:12"  
}
```

6. Mensageria (RabbitMQ)

O agendamento-service publica eventos de consulta em um exchange do tipo **topic**, consumidos de forma independente por dois serviços diferentes, um mesmo evento é entregue a ambos os consumidores.

Elemento	Nome	Descrição
Exchange	consulta.exchange	Tipo <i>topic</i> , ponto único de publicação de eventos de consulta
Routing key	consulta.criada	Publicada quando uma nova consulta é cadastrada
Routing key	consulta.editada	Publicada quando uma consulta é atualizada
Fila	notificacao.consulta.queue	Consumida pelo notificacao-service
Fila	historico.consulta.queue	Consumida pelo historico-service
Padrão de binding	consulta.*	Ambas as filas recebem tanto eventos de criação quanto de edição

Ao consumir um evento:

- O **notificacao-service** registra em log o envio de um lembrete ao paciente (sem integração real de e-mail/SMS, por decisão de escopo do desafio).
- O **historico-service** grava ou atualiza a projeção de leitura correspondente (registrarOuAtualizar), identificando o registro existente pelo consultId do evento, o mesmo fluxo trata tanto a criação quanto a edição de uma consulta.

A topologia (exchange, filas e *bindings*) é pré-carregada na subida do broker via `docker/rabbitmq/definitions.json`, e não é criada pela aplicação em tempo de execução, isso garante que a infraestrutura de mensageria já exista antes de qualquer serviço tentar se conectar a ela.

7. Documentação Swagger

7.1 Documentação da API com Swagger

O agendamento-service possui documentação interativa gerada automaticamente utilizando o **SpringDoc OpenAPI (Swagger)**, permitindo visualizar e testar os endpoints REST sem a necessidade de ferramentas externas.

Após iniciar a aplicação, a documentação pode ser acessada em:

<http://localhost:3710/swagger-ui.html>

Por meio da interface do Swagger é possível:

- Visualizar todos os endpoints disponibilizados pela API, organizados por controller
- Consultar os DTOs e VOs utilizados em cada operação, com descrição e exemplo de cada campo
- Ver os parâmetros de entrada e os modelos de requisição e resposta
- Identificar os códigos de status HTTP retornados por cada endpoint, incluindo os erros no padrão Problem Details
- Autenticar-se com um token JWT (botão **Authorize**) e executar requisições diretamente pela interface ("Try it out")

Como a documentação é gerada automaticamente a partir do código da aplicação, ela permanece sincronizada com os endpoints implementados, facilitando o desenvolvimento, os testes e a integração com outros sistemas.

7.2 Exemplos de Interface

Tela Inicial

A tela inicial apresenta todos os controllers e endpoints organizados por categoria.

 Swagger
OPENAPI SPECIFICATION

/v3/api-docs

Explore



PulseCare API - Agendamento Service

1.0.0

OA5.3.1

[/v3/api-docs](#)

API para cadastro de usuários e agendamento de consultas médicas

Servers

http://localhost:3710 - Generated server url

Authorize



Consulta

Cadastro, edição, busca e listagem de consultas médicas

GET

/consultas/{id}

Busca consulta por id





PUT

/consultas/{id}

Atualizar consulta





GET

/consultas

Lista consultas por paciente ou médico





POST

/consultas

Cadastrar consulta





Tipo Usuário

Consulta dos papéis (roles) disponíveis na plataforma

GET

/tipos-usuario

Lista todos os tipos de usuário





GET

/tipos-usuario/{id}

Busca tipo de usuário por id





Usuário

Cadastro e gestão de usuários (médicos, enfermeiros e pacientes)

Usuário

Cadastro e gestão de usuários (médicos, enfermeiros e pacientes)

GET

/usuarios/{id}

Busca usuário por id





PUT

/usuarios/{id}

Atualizar usuário





DELETE

/usuarios/{id}

Deletar usuário





GET

/usuarios

Lista usuários de forma paginada





POST

/usuarios

Cadastrar usuário





PATCH

/usuarios/{id}/senha

Alterar senha





Auth

Login único da plataforma, válido em todos os módulos

POST

/auth/login

Login





Schemas

UsuarioUpdateVO > Expand all object

TipoUsuarioDTO > Expand all object

UsuarioDTO > Expand all object

ProblemDetailDTO > Expand all object

ConsultaUpdateVO > Expand all object

UsuarioUpdateVO > Expand all object

TipoUsuarioDTO > Expand all object

UsuarioDTO > Expand all object

ProblemDetailDTO > Expand all object

ConsultaUpdateVO > Expand all object

ConsultaDTO > Expand all object

UsuarioVO > Expand all object

ConsultaVO > Expand all object

LoginVO > Expand all object

LoginResponseDTO > Expand all object

AlterarSenhaVO > Expand all object

PageUsuarioDTO > Expand all object

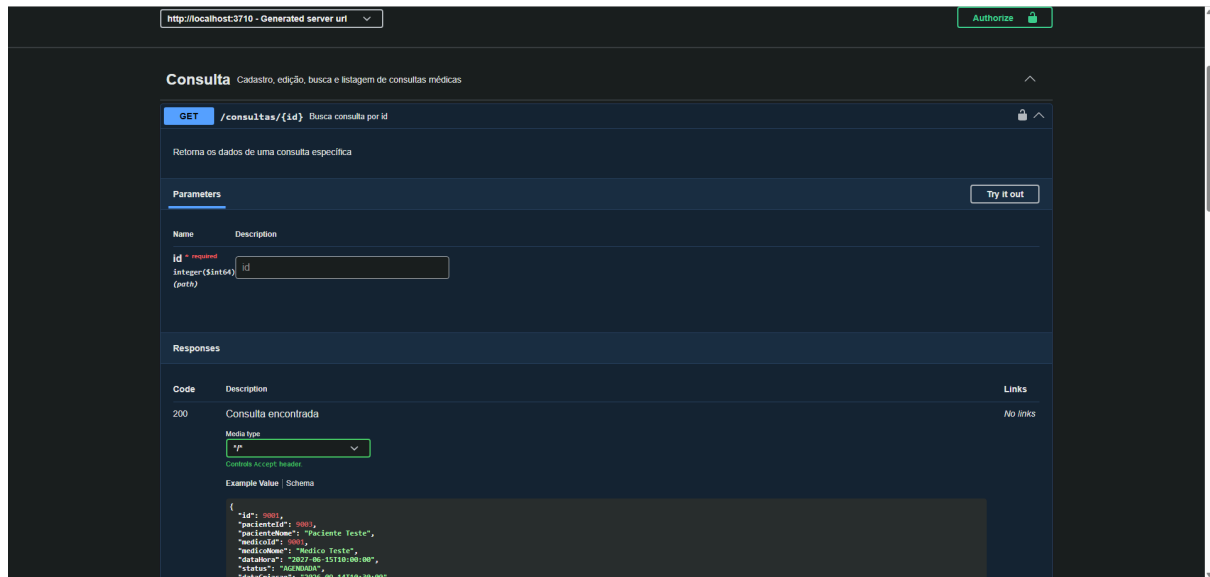
PageableObject > Expand all object

SortObject > Expand all object

PageConsultaDTO > Expand all object

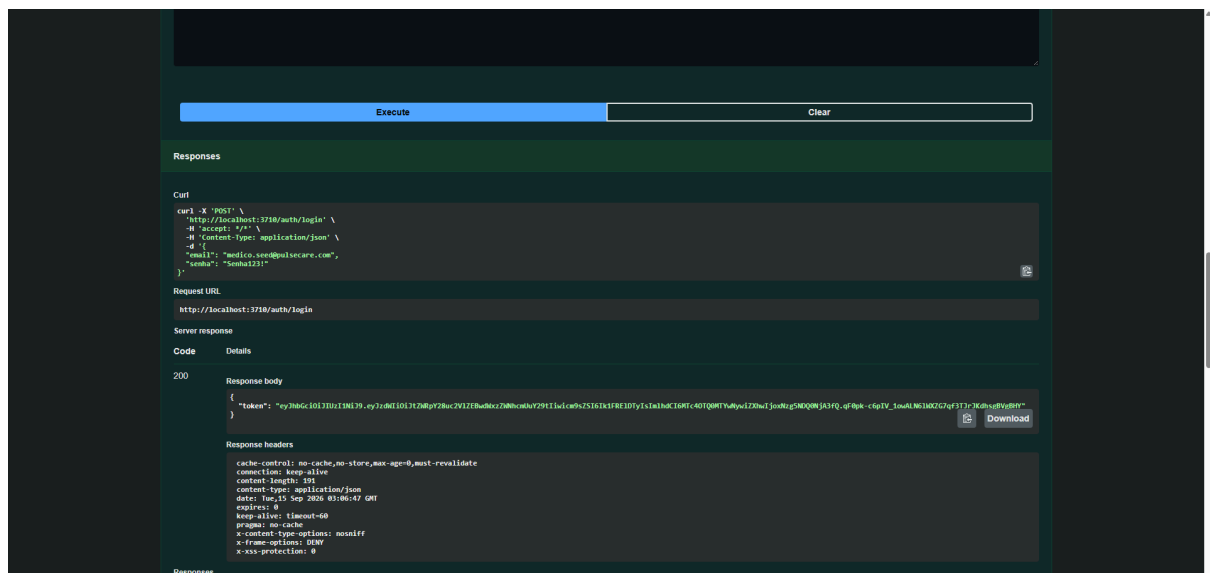
Exemplo de Endpoint Expandido

Ao expandir um endpoint, é possível visualizar detalhes da requisição, incluindo parâmetros, corpo da requisição e exemplos de resposta.



Execução de requisição

A funcionalidade "Try it out" permite testar os endpoints diretamente pela interface.



6. Documentação Postman

6.1 Testes da API com Postman

Para facilitar os testes e validações dos endpoints do agendamento-service, foi disponibilizada uma **collection do Postman** contendo requisições prontas para os principais fluxos da aplicação.

Arquivo da Collection

- Nome: Pulse Care.postman_collection.json
- Localização: raiz do projeto

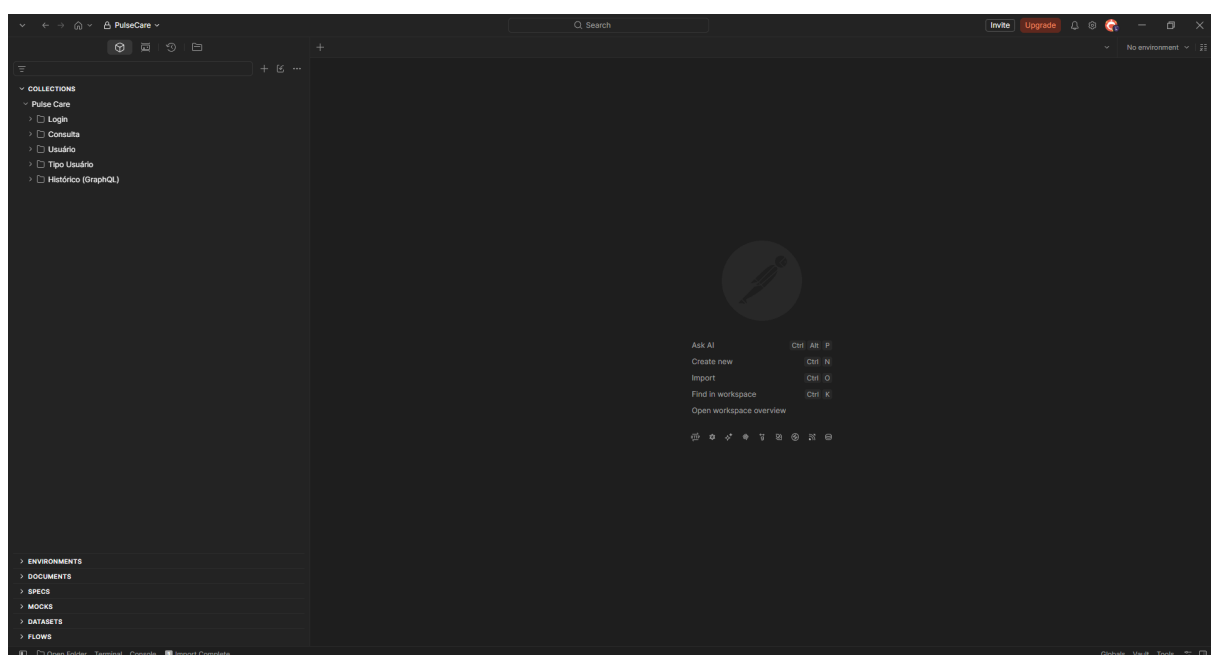
A requisição de **Login** define automaticamente a variável de coleção {{token}} por meio de um script de teste, reaproveitada nas demais requisições da collection.

Uma massa de dados fixa (V002__seed_dados_teste.sql, com IDs a partir de 9001) é aplicada pelo Flyway em toda subida do banco de dados, permitindo executar os cenários de sucesso em qualquer ordem, sem depender do resultado de uma requisição anterior. Os usuários de teste seguem a senha padrão Senha123!.

6.2 Exemplos da Interface

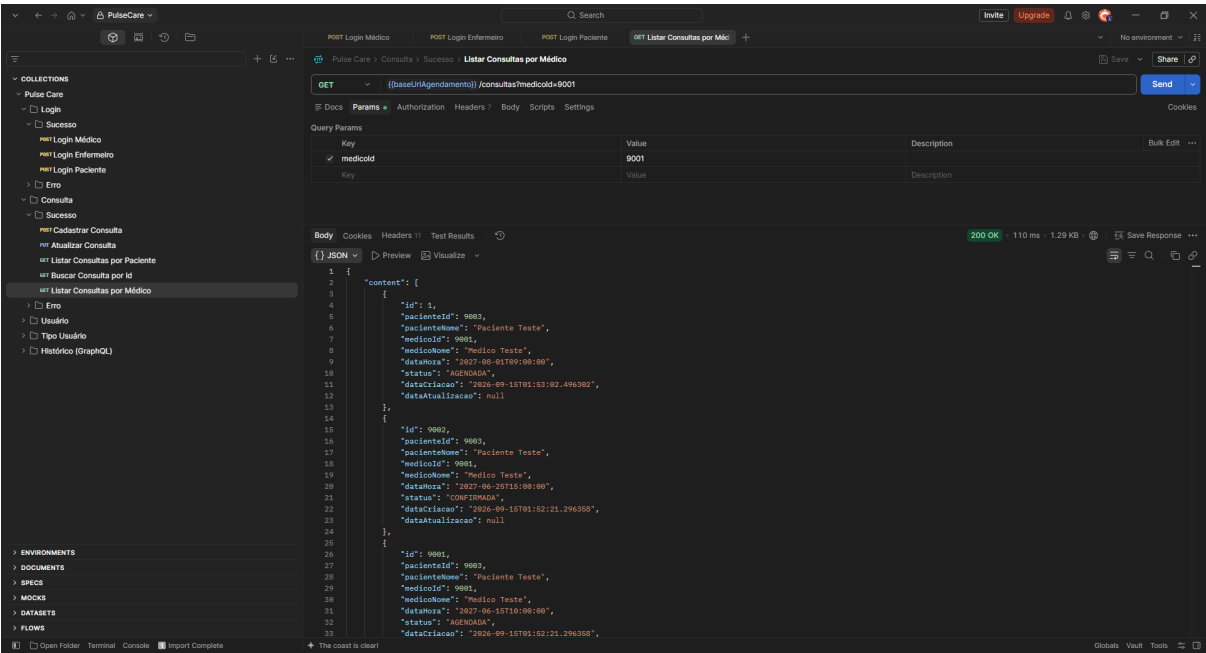
Collection importada

Visualização da collection com todos os endpoints organizados por categoria.



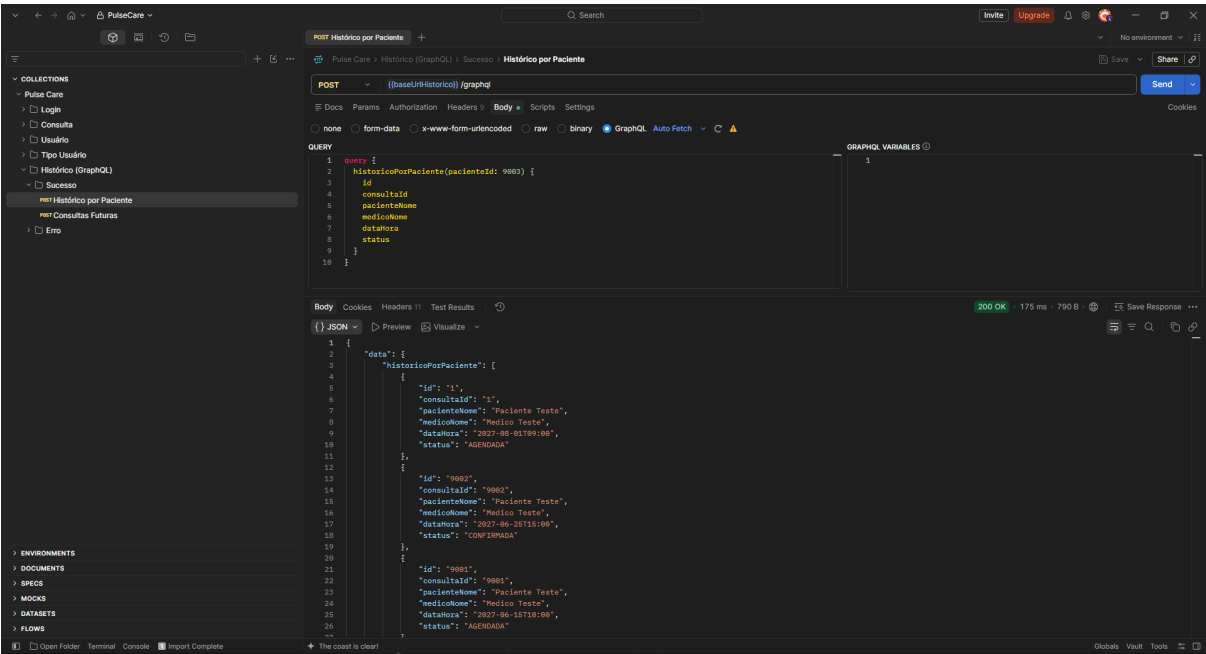
Execução de requisição

Exemplo de execução de uma requisição com retorno da API.



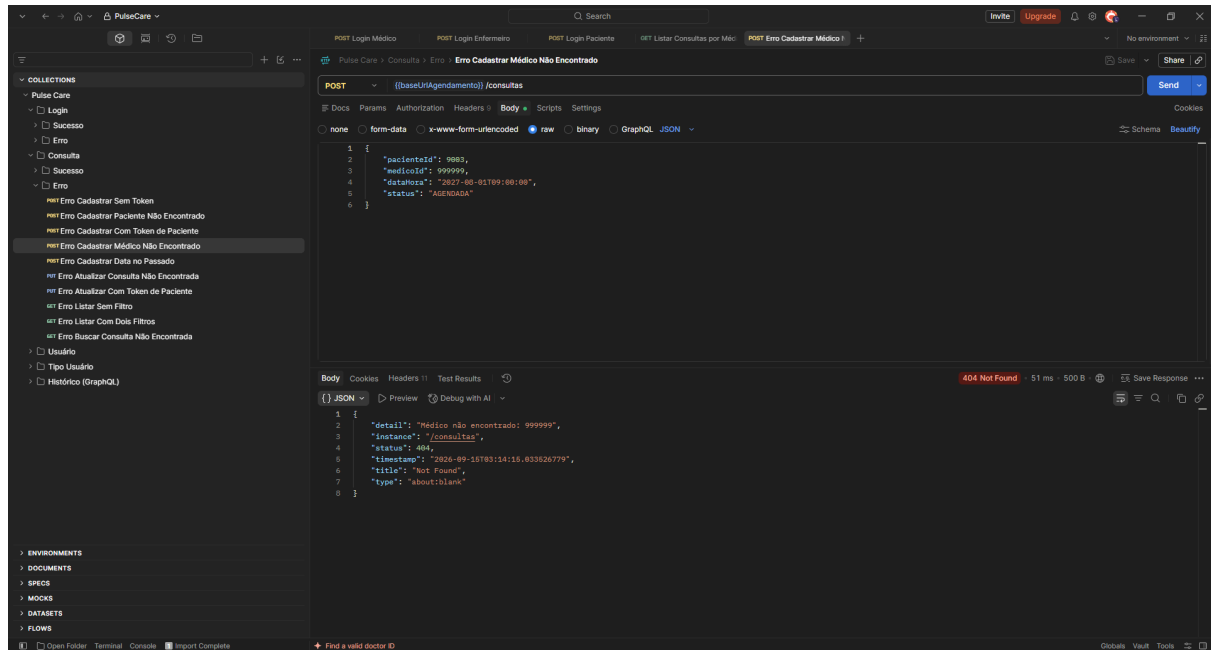
Execução de requisição GraphQL

Exemplo de execução de uma requisição GraphQL.



Separação em Sucesso e Error

As requisições estão organizadas por domínio (Tipo Usuário, Usuário, Consulta e Historico/GraphQL) e para cada um possuem pastas, com massas de testes prontas para as execuções de sucesso e error



9. Estrutura do Banco de Dados

A persistência é feita em dois bancos PostgreSQL independentes, seguindo o padrão *database-per-service*, ambos versionados via **Flyway**:

agendamento_db (fonte da verdade, escrita via REST) - tipo_usuario - usuario - consulta

historico_db (projeção de leitura, escrita apenas via evento assíncrono) - historico_consulta

Os dois bancos compartilham a mesma instância do PostgreSQL (um único container), mas são databases logicamente separados - nenhuma *query* cruza os dois bancos, reforçando o desacoplamento entre os serviços.

10. Execução da Aplicação com Docker Compose

10.1 Clonar o repositório

```
git clone https://github.com/LuisFelipeGM/PulseCare.git
cd PulseCare
```

10.2 Configurar variáveis de ambiente

O projeto utiliza variáveis de ambiente para configuração do banco de dados e da aplicação. Criar arquivo .env:

Copie o arquivo de exemplo:

```
cp .env.example .env
```

No Windows (PowerShell):

```
copy .env.example .env
```

O Docker Compose carrega o .env automaticamente e usa esses valores para configurar credenciais do banco, do RabbitMQ, o segredo do JWT e as portas expostas de cada serviço. Os valores do .env.example já funcionam para rodar localmente sem qualquer ajuste.

10.3 Executar a aplicação

```
docker compose up --build
```

Esse comando sobe o Postgres, o RabbitMQ e os três microsserviços, aplicando as migrations do Flyway automaticamente em cada serviço.

10.4 Acessar a aplicação

Serviço	URL
agendamento-service (API + Swagger)	http://localhost:3710
historico-service (GraphQL)	http://localhost:3711
notificacao-service	Sem API HTTP pública (Verificar logs do container)
RabbitMQ Management	http://localhost:15672

Observações importantes

- O arquivo .env não é versionado por conter dados sensíveis; utilize o .env.example como base.
- Os serviços se conectam ao banco e ao broker utilizando o nome do container como hostname (postgres, rabbitmq).

11. Repositório do Projeto

URL do Repositório:

<https://github.com/LuisFelipeGM/PulseCare>